FANCODE ENGINEERING ASSESSMENT

Live Sports Match Data Aggregator

Comprehensive System Documentation & Technical Specification

Technology Stack: Spring Boot 3.3.5, Java 21 LTS, Spring Data JPA

Persistence: Persistent Embedded H2 Database (File Mode)

Test Suite: 11/11 Automated Tests Passing (100% Success)

Deployment: Multi-Stage Docker & Standalone Maven Wrapper

Author: Deepak Agrawal

Repository: `C:\Users\Deepak\OneDrive\Desktop\fancode-aggregator`

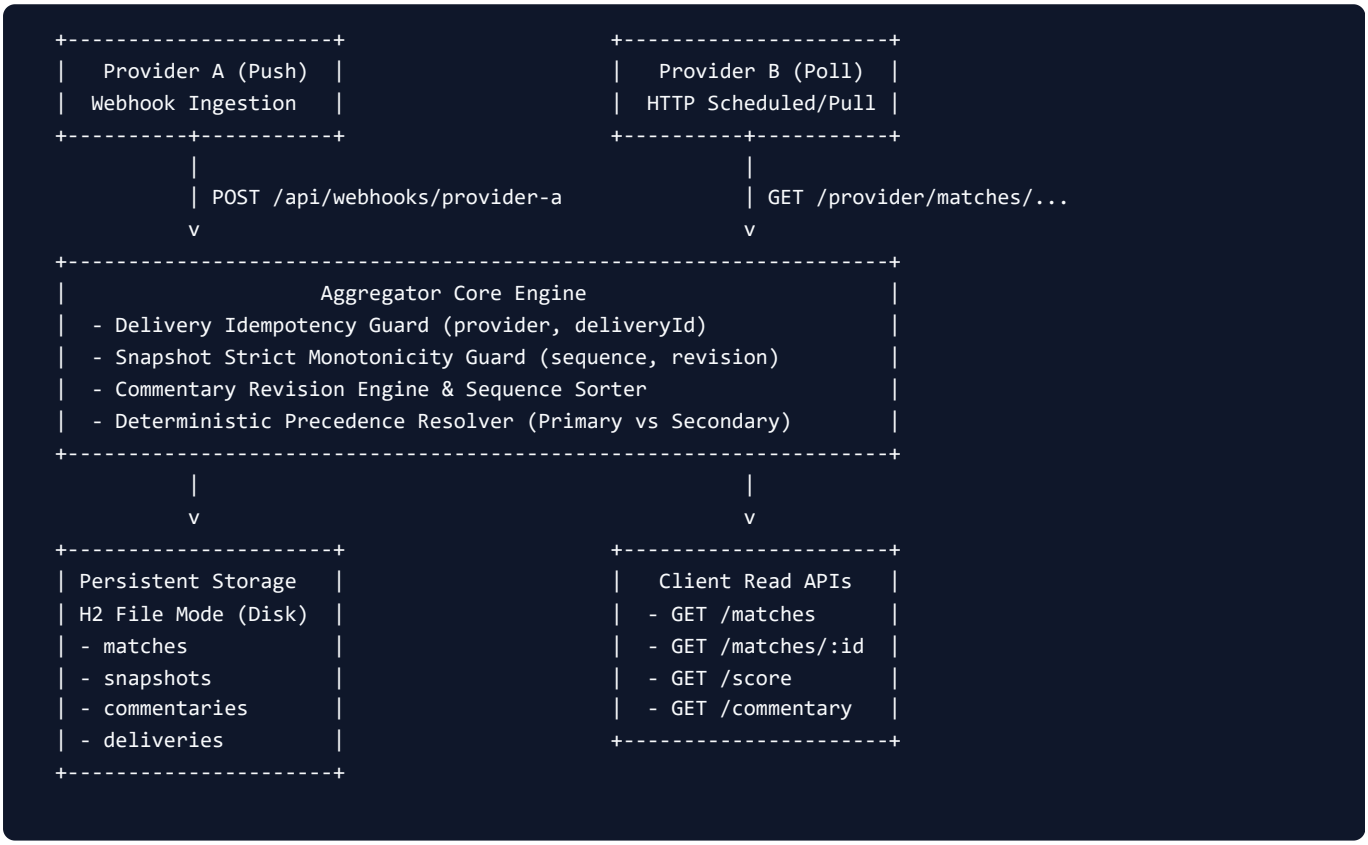
Date: November 2026

1. Executive Summary & System Overview

The **Live Sports Match Data Aggregator** is an enterprise-grade backend service engineered to ingest live cricket match data concurrently from two distinct sports data providers and expose a unified, consistent, and highly performant set of client read APIs.

> **Core Challenge:** Real-world sports data feeds suffer from network retries, transmission delays, duplicate messages, out-of-order packets, commentary corrections, and independent numbering systems. The aggregator resolves these complexities without ever computing runs or interpreting commentary text.

High-Level Architecture



Key Engineering Principles

- **Opaque Snapshot Interpretation:** Scores (e.g. `98/2`) and overs (e.g. `"12.4"`) are treated as authoritative snapshot facts from the provider, never computed by parsing commentary.
- **Lexicographical Monotonicity:** A score snapshot `(sequence, revision)` is applied if and only if it is strictly greater than the currently stored snapshot for that provider.
- **Commentary Sequence Independence:** Commentary display order is governed strictly by the ball's `sequence` number, not packet arrival time. Out-of-order deliveries and missing sequence gaps are seamlessly accommodated without delaying score updates.
- **Cross-Provider Isolation:** Sequence numbers are strictly provider-specific and are **never** compared across providers.

2. Domain Model & Relational Schema

The persistence model decouples provider streams into independent storage partitions, merging them on-demand via deterministic precedence.

Table Name	Primary Key	Key Fields & Constraints	Purpose
`matches`	`match_id` (VARCHAR)	`status`, `active_provider`, `updated_at`	Tracks canonical match identity (`match-1001`) and high-level status.
`provider_deliveries`	`id` (BIGINT)	`provider`, `delivery_id` **UNIQUE:** `(provider, delivery_id)`	Fast-path deduplication table ensuring push webhook idempotency.
`provider_snapshots`	`id` (BIGINT)	`sequence_num`, `revision_num`, `runs`, `wickets`, `overs` **UNIQUE:** `(match_id, provider)`	Stores the latest validated scoreboard snapshot for each provider independently.
`provider_commentaries`	`id` (BIGINT)	`ball_id`, `sequence_num`, `revision_num`, `text` **UNIQUE:** `(match_id, provider, ball_id)` **INDEX:** `(match_id, provider, sequence_num)`	Stores ball-by-ball commentary items, preserving stable ball identity for corrections.

Monotonicity & Ordering Contracts

Scoreboard Snapshot Rule

A snapshot candidate `(s2, r2)` is strictly newer than stored `(s1, r1)` iff:

```
(s2 > s1) || (s2 == s1 && r2 > r1)
```

Example: (5, 1) > (4, 9); (4, 2) > (4, 1); (4, 1) is NOT newer than (4, 1).

Commentary Revision Rule

A ball update with `ballId` is accepted iff incoming revision exceeds stored revision:

```
candidate.revision > stored.revision
```

Equal or lower revisions leave stored text untouched. Total ball count remains exactly 1.

3. Ingestion Pipelines: Push vs. Poll

Provider A: Push Webhook Pipeline

Provider A delivers real-time events via `POST /api/webhooks/provider-a`. Each message is tagged with an envelope containing `deliveryId`, `occurredAt`, and optional `snapshot` and `commentary` payloads.

- **Idempotency Verification:** The service queries `provider_deliveries`. If the `deliveryId` already exists, processing immediately terminates with HTTP 200 and body `{"status": "duplicate_ignored"}`. Stored data is unchanged.
- **Atomic Record & Process:** If new, the delivery ID is persisted, the canonical match ID (`match-1001`) is resolved from `a-match-1001`, and snapshots and commentary are evaluated under a transactional lock.

Provider B: Polling Pipeline

Provider B exposes pull endpoints: `GET /provider/matches/:id/score` and `GET /provider/matches/:id/commentary`.

- **Scheduled Execution:** Background task runs periodically (default 5000ms), querying Provider B score and commentary.
- **Fault Tolerance & Graceful Degradation:** If Provider B responds with HTTP 503, connection timeouts, or socket failures, the poller catches the exception cleanly, logs an alert, and **preserves the last good stored state**. The client read API continues serving without disruption.
- **Deduplication Without Delivery IDs:** Repeated identical polls evaluate against stored snapshot tuples and commentary revisions. Because incoming revisions equal existing revisions, no duplicate records or state changes occur.

Push Worked Scenario Execution Trace

Step	Message	Payload Ingested	Scoreboard State	Commentary State	Outcome / Reason
1	`d1`	Snap (1, 1)=4/0 (0.4), Ball b1 rev 1 "four"	4/0 (0.4)	[b1]	Initial baseline applied.
2	`d2`	Snap (2, 1)=4/1 (0.5), Ball b2 rev 1 "out"	4/1 (0.5)	[b1, b2]	(2, 1) > (1, 1). Score and new ball applied.
3	`d2`	Snap (2, 1)=4/1 (0.5), Ball b2 rev 1 "out"	4/1 (0.5)	[b1, b2]	**duplicate_ignored**. Same deliveryId. State unchanged.
4	`d4`	Snap (4, 1)=10/1 (1.1), Ball b4 rev 1 "six"	10/1 (1.1)	[b1, b2, b4]	Out of order arrival. Score updated immediately. Gap permitted.
5	`d3`	Snap (3, 1)=8/1 (1.0), Ball b3 rev 1 "two runs"	10/1 (1.1)	[b1, b2, b3, b4]	Snap (3, 1) < (4, 1) ignored. Late ball b3 inserted in order!
6	`d5`	Snap (4, 1)=10/1 (1.1), Ball b2 rev 2 "out, review stays out"	10/1 (1.1)	[b1, b2*, b3, b4]	Snap (4, 1) == (4, 1) ignored. Ball b2 rev 2 > rev 1: text corrected!

4. Client Read APIs & Precedence Rules

Deterministic Precedence Rule

The service is configured with a Primary Provider (`app.primary-provider = provider-a`).

- When client requests match scores or commentary, the aggregator inspects the primary provider's partition first.
- If Provider A has a valid snapshot, Provider A's score and commentary are served.
- If Provider A has not published data or is in an uninitialized state, the system automatically falls back to Provider B.
- **Crucial Invariant:** Commentary is never merged or interweaved between providers. It always reflects the single active provider, sorted chronologically by sequence number.

Client API Endpoints

1. List All Matches: `GET /matches`

Returns summary list of all known matches with current status and summary score.

```
HTTP/1.1 200 OK
[
  {
    "id": "match-1001",
    "status": "live",
    "summaryScore": "10/1 (1.1)"
  }
]
```

2. Get Match Scoreboard: `GET /matches/:id/score`

Returns current score snapshot from the winning provider.

```
HTTP/1.1 200 OK
{
  "status": "live",
  "innings": 1,
  "runs": 10,
  "wickets": 1,
  "overs": "1.1",
  "scoreFormatted": "10/1",
  "provider": "provider-a"
}
```

3. Get Match Commentary: `GET /matches/:id/commentary`

Returns ball-by-ball commentary strictly ordered by sequence number.

```
HTTP/1.1 200 OK
[
  { "ballId": "b1", "sequence": 1, "revision": 1, "text": "four" },
  { "ballId": "b2", "sequence": 2, "revision": 2, "text": "out, review stays out" },
  { "ballId": "b3", "sequence": 3, "revision": 1, "text": "two runs" },
```

```
  { "ballId": "b4", "sequence": 4, "revision": 1, "text": "six" }
]
```

5. Automated Test Suite & Verification

The automated test suite contains 11 comprehensive tests in JUnit 5 and MockMvc, rigorously covering every specification requirement.

> **Verification Result:** 11 tests executed, 11 passed, 0 failures, 0 errors. All test runs execute in under 18 seconds via Maven.

Test Name	Target Requirement	Verification Logic
`testOrderedIngestion`	Ordered Delivery	Ingests d1 then d2. Verifies score transitions 4/0 -> 4/1 and commentary has 2 items in sequence.
`testDuplicatePushDelivery`	Duplicate Push Delivery	Sends d1 twice. Second request returns `duplicate_ignored`. DB ball count stays exactly 1.
`testOlderSnapshotWithLateBallReturnedInSequence`	Out-of-Order & Gap Handling	d4 arrives before d3. Verifies score 10/1 is preserved when d3 arrives, while ball b3 is inserted between b2 and b4.
`testCommentaryCorrection`	Commentary Revision	Ball b2 rev 2 replaces text "out" with "out, review stays out". Sending rev 1 afterwards is ignored.
`testRepeatedIdenticalPolls`	Poll Idempotency	Polls Provider B multiple times in "unchanged" state. Zero duplicate balls created in the database.
`testStalePoll`	Stale Poll Rejection	Polls sequence 16 (104/2), then polls stale sequence 10 (80/1). Score remains 104/2.
`testProviderErrorLeavesLastGoodState`	Provider Fault Tolerance	Simulates HTTP 503 from Provider B. Poller catches error gracefully, score remains 98/2.
`testRestartAgainstSameDatabase`	Durable Disk Persistence	Populates DB in Application Context 1, shuts it down completely, boots Application Context 2 against disk file, and verifies full state recovery.
`testFullWorkedPushScenario`	Worked Spec Example	Executes full 6-step sequence (d1, d2, d2, d4, d3, d5). Final score is 10/1, commentary has all 4 balls.
`testUnknownMatchReturns404`	Error Handling Contract	Requests for unknown match IDs return HTTP 404 with structured JSON.
`testWebhookAndClientReadFlow`	End-to-End Webhook API	Validates HTTP POST /api/webhooks/provider-a and subsequent GET requests via Spring MockMvc.

Execution Instructions

```
# Run all automated tests:
.\mvnw.cmd test

# Run application locally:
.\mvnw.cmd spring-boot:run

# Run in containerized environment:
docker compose up --build
```